PROFESSOR DODGE

Theoretical Guide: Angular, Spring Boot & Software Engineering

Comprehensive Answers to Professor-Level Questions
Generated: 2026-05-04 22:53
SMGO Content Management System Case Study

TABLE OF CONTENTS

1. ANGULAR FRAMEWORK THEORY

1.1 Component Architecture

Question 1: What is the component-based architecture in Angular and how does it differ from traditional MVC?

Angular's component-based architecture treats components as the fundamental building blocks of applications. Unlike traditional MVC where controllers manage views, Angular components encapsulate both the view (template) and the logic (component class), creating self-contained, reusable units. Key differences from MVC: • Components combine view + logic in single units • Hierarchical component tree replaces flat MVC structure • Data binding provides automatic synchronization • Components promote reusability and maintainability In SMGO project, components like 'unified-home', 'ai-discovery', and 'admin-notifications' demonstrate this architecture.

Question 2: Explain the lifecycle hooks of an Angular component and their execution order.

Angular components have 8 lifecycle hooks executed in this order: 1. ngOnChanges() - When input properties change 2. ngOnInit() - After component initialization 3. ngDoCheck() - Custom change detection 4. ngAfterContentInit() - After content projection 5. ngAfterContentChecked() - After content checking 6. ngAfterViewInit() - After view initialization 7. ngAfterViewChecked() - After view checking 8. ngOnDestroy() - Before component destruction Example from SMGO: ```typescript export class UnifiedHomeComponent implements OnInit, OnDestroy { ngOnInit() { this.loadUserContent(); this.initializeWebSocket(); } ngOnDestroy() { this.websocketSubscription.unsubscribe(); } } ```

Question 3: How does Angular handle component communication (parent-child, child-parent, sibling components)?

Angular provides several communication patterns: 1. Parent to Child: @Input() decorator ```typescript // Parent component // Child component @Input() data: any; ``` 2. Child to Parent: @Output() with EventEmitter ```typescript // Child component @Output() dataChanged = new EventEmitter(); // Parent component ``` 3. Sibling Communication: Shared service with Subject ```typescript @Injectable() export class SharedService { private dataSubject = new Subject(); data$ = this.dataSubject.asObservable(); sendData(data: any) { this.dataSubject.next(data); } } ``` 4. ViewChild/ContentChild for direct access

Question 4: What are Angular directives and how do they extend HTML functionality?

Directives are classes that extend HTML functionality by attaching behavior to elements. Angular has 3 types: 1. Components - directives with templates 2. Attribute directives - change element appearance/behavior 3. Structural directives - change DOM layout Built-in directives: • *ngIf - conditional rendering • *ngFor - list iteration • *ngSwitch - conditional switching • [ngClass] - dynamic CSS classes • [ngStyle] - dynamic styles Custom directive example: ```typescript @Directive({ selector: '[appHighlight]' }) export class HighlightDirective { @HostListener('mouseenter') onMouseEnter() { this.highlight('yellow'); } private highlight(color: string) { this.el.nativeElement.style.backgroundColor = color; } } ```

Question 5: Explain the difference between structural and attribute directives with examples.

Structural directives change DOM structure by adding/removing elements: • *ngIf - conditionally includes/excludes elements • *ngFor - repeats elements for each item in collection • *ngSwitch - displays one element from set of options Attribute directives change element appearance/behavior without changing structure: • [ngClass] - adds/removes CSS classes • [ngStyle] - sets inline styles • [disabled] - sets disabled attribute Example usage in SMGO: ```html {{content.title}} Submit ```

1.2 Reactive Programming & RxJS

Question 1: What is reactive programming and how does RxJS implement it in Angular?

Reactive programming is a paradigm focused on data streams and propagation of change. RxJS (Reactive Extensions for JavaScript) brings reactive programming to Angular through Observables. Key concepts: • Observable: Represents stream of data over time • Observer: Consumes data from Observable • Subscription: Connection between Observable and Observer • Operators: Functions to transform, filter, combine streams Angular uses RxJS extensively: • HttpClient returns Observables • Event handling with fromEvent • Reactive forms with FormControl.valueChanges • Router events and guards Benefits: • Declarative handling of asynchronous operations • Composable operators for complex data flows • Automatic unsubscription management • Better error handling and retry logic

Question 2: Explain the Observer pattern and how Observables work in Angular.

The Observer pattern defines relationship between Observable (subject) and Observer (subscriber): Observable emits data through three methods: • next(value) - emits new value • error(error) - emits error • complete() - signals completion Observer subscribes to receive these emissions: ```typescript interface Observer { next: (value: T) => void; error: (err: any) => void; complete: () => void; } ``` Subscription manages the connection: ```typescript const subscription = observable.subscribe({ next: (value) => console.log(value), error: (err) => console.error(err), complete: () => console.log('Done') }); // Cleanup subscription.unsubscribe(); ``` In SMGO, WebSocket notifications use this pattern: ```typescript this.notificationService.getNotifications().subscribe({ next: (notification) => this.showNotification(notification), error: (error) => this.handleError(error) }); ```

Question 3: What are the main RxJS operators used in Angular applications?

RxJS operators transform, filter, and combine Observables: Creation operators: • of() - creates Observable from values • from() - converts array/promise to Observable • interval() - emits sequential numbers • fromEvent() - creates from DOM events Transformation operators: • map() - transforms emitted values • switchMap() - maps to inner Observable, switches • mergeMap() - maps to inner Observable, merges • concatMap() - maps to inner Observable, concatenates Filtering operators: • filter() - emits values that pass predicate • take() - takes first N emissions • takeUntil() - takes until another Observable emits • debounceTime() - emits value after pause Combination operators: • combineLatest() - combines latest values • forkJoin() - waits for all to complete • zip() - combines corresponding emissions • merge() - merges multiple Observables Example in SMGO AI recommendations: ```typescript this.userPreferences$.pipe(debounceTime(300), switchMap(prefs =>

this.aiService.getRecommendations(prefs)), catchError(error => of([]))).subscribe(recommendations => this.displayRecommendations(recommendations)); ```

Question 4: How does Angular handle asynchronous operations with Observables?

Angular handles async operations through Observable streams: 1. HTTP Requests: ```typescript getContents(): Observable { return this.http.get('/api/contents').pipe(catchError(this.handleError)); } ``` 2. Reactive Forms: ```typescript this.searchForm.get('query').valueChanges.pipe(debounceTime(300), distinctUntilChanged(), switchMap(query => this.searchService.search(query))).subscribe(results => this.searchResults = results); ``` 3. Route Guards: ```typescript canActivate(): Observable { return this.authService.isAuthenticated().pipe(map(isAuth => { if (!isAuth) this.router.navigate(['/login']); return isAuth; })); } ``` 4. Component lifecycle with async data: ```typescript ngOnInit() { this.loading$ = of(true); this.contentService.getTop5Contents().pipe(finalize(() => this.loading$ = of(false))).subscribe(contents => this.topContents = contents); } ``` Benefits over callbacks/promises: • Cancellation with unsubscribe() • Multiple subscribers • Rich operator ecosystem • Declarative error handling

Question 5: Explain the difference between Promises and Observables.

Promises vs Observables: Promises: • Single future value • Eager execution (starts immediately) • Cannot be cancelled • One-time resolution (fulfilled/rejected) • No operators for transformation Observables: • Stream of values over time • Lazy execution (starts on subscribe) • Can be cancelled (unsubscribe) • Multiple emissions (next/error/complete) • Rich operator ecosystem When to use each: Promises for: • Single async operation • Simple success/failure scenarios • Interop with non-RxJS code Observables for: • Multiple async values • Complex data transformations • Cancellation requirements • Event streams • HTTP requests in Angular SMGO example: ```typescript // Promise - single authentication check authenticate(credentials): Promise { return this.http.post('/auth/login', credentials).toPromise(); } // Observable - real-time notifications getNotifications(): Observable { return this.websocketService.connect().pipe(filter(message => message.type === 'notification'), map(message => message.payload)); } ```

1.3 Dependency Injection

Question 1: Explain Angular's dependency injection system and its benefits.

Dependency Injection (DI) is a design pattern where dependencies are provided to components rather than created internally. Angular's DI system manages object creation and injection automatically. Benefits: • Loose coupling between components • Easier testing with mock dependencies • Reusable services across components • Centralized configuration • Better code maintainability Core concepts: • Injector - container that holds dependencies • Provider - recipe for creating dependencies • Token - identifier for dependency • Dependency - object to be injected Example: ```typescript @Injectable({ providedIn: 'root' }) export class ContentService { constructor(private http: HttpClient) {} } @Component({...}) export class ContentListComponent { constructor(private contentService: ContentService) {} } ```

Question 2: What are the different types of providers in Angular DI?

Angular provides several provider types: 1. Class Provider: ```typescript providers: [{ provide: ContentService, useClass: ContentService }] ``` 2. Value Provider: ```typescript providers: [{ provide: 'API_URL', useValue: 'http://localhost:8090' }] ``` 3. Factory Provider: ```typescript providers: [{ provide: ContentService, useFactory: (http: HttpClient) => new ContentService(http, 'custom-config'), deps: [HttpClient] }] ``` 4. Existing Provider: ```typescript providers: [{ provide: NewService, useExisting: OldService }] ``` Modern approach (Angular 6+): ```typescript @Injectable({ providedIn: 'root' // singleton }) @Injectable({ providedIn: 'any' // new instance per injector }) ```

Question 3: How does hierarchical dependency injection work in Angular?

Angular's DI is hierarchical, with injectors at different levels: 1. Platform Injector - root level, shared across app 2. Root Injector - NgModule level 3. Component Injector - component level 4. Element Injector - directive/element level Resolution hierarchy: 1. Component's element injector 2. Parent component injectors 3. Root injector 4. Platform injector Benefits: • Component-specific services • Lazy loading module isolation • Override services in specific contexts Example: ```typescript // Root service (singleton) @Injectable({ providedIn: 'root' }) export class GlobalService {} // Component-specific service @Component({ providers: [LocalService] }) export class MyComponent { constructor(private local: LocalService, private global: GlobalService) {} } ```

Question 4: Explain the difference between singleton and transient services.

Singleton Services: • One instance shared across entire application • Created once, reused everywhere • Provided in 'root' or NgModule providers • Good for shared state, caching, HTTP services Transient Services: • New instance created each time injected • Isolated state per component • Provided in component providers or 'any' • Good for component-specific data Examples: Singleton (SMGO AI Service): ```typescript @Injectable({ providedIn: 'root' }) export class AiRecommendationService { private model: any; constructor() { this.loadModel(); // Load once for all components } } ``` Transient (Component-specific): ```typescript @Injectable() export class ComponentStateService { data: any = {}; // Fresh instance per component } @Component({ providers: [ComponentStateService] }) export class MyComponent { constructor(private state: ComponentStateService) {} } ```

2. SPRING BOOT THEORY

2.1 Inversion of Control & Dependency Injection

Question 1: Explain Inversion of Control (IoC) and how Spring implements it.

Inversion of Control (IoC) is a design principle where control flow is inverted compared to traditional programming. Instead of application code controlling dependencies, the framework (Spring) controls them. Traditional approach (tight coupling): ```java public class ContentService { private DatabaseRepository repo = new DatabaseRepository(); } ``` IoC with Spring: ```java @Service public class ContentService { private final DatabaseRepository repo; @Autowired public ContentService(DatabaseRepository repo) { this.repo = repo; } } ``` Spring IoC Container: • BeanFactory - basic container • ApplicationContext - advanced container with features • Manages bean lifecycle (creation, initialization, destruction) • Provides DI, AOP, event handling Benefits: • Loose coupling • Easier testing • Better maintainability • Centralized configuration

Question 2: What is the difference between tight coupling and loose coupling?

Tight Coupling: • Direct instantiation of dependencies • Hard dependencies between classes • Difficult to test and maintain • Changes ripple through codebase Loose Coupling with DI: • Dependencies injected from outside • Interfaces used instead of concrete classes • Easy to mock for testing • Flexible configuration ```java @Service public class NotificationService { private final NotificationSender sender; @Autowired public NotificationService(NotificationSender sender) { this.sender = sender; // Loose coupling } } ``` In SMGO project: ```java // Interface public interface NotificationSender { void send(String message); } // Implementations @Service @Qualifier("email") public class EmailSender implements NotificationSender {...} @Service @Qualifier("firebase") public class FirebaseSender implements NotificationSender {...} // Usage @Autowired @Qualifier("firebase") private NotificationSender firebaseSender; ```

Question 3: How does Spring's dependency injection work at runtime?

Spring DI works through several phases: 1. Bean Definition Scanning: • @ComponentScan scans for @Component, @Service, @Repository, @Controller • XML configuration or JavaConfig classes • Bean definitions registered in ApplicationContext 2. Bean Instantiation: • Constructor injection (preferred) • Setter injection • Field injection (not recommended) 3. Dependency Resolution: • Circular dependency detection • BeanPostProcessor for customization • @PostConstruct for initialization 4. Bean Lifecycle: • Bean creation and wiring • Initialization (@PostConstruct) • Ready for use • Destruction (@PreDestroy) Runtime example: ```java @Configuration public class AppConfig { @Bean public ContentService

contentService(DatabaseRepository repo) { return new ContentService(repo); } @Bean public DatabaseRepository databaseRepository() { return new MongoRepository(); } } // At runtime: ApplicationContext context = SpringApplication.run(AppConfig.class); ContentService service = context.getBean(ContentService.class); ```

Question 4: Explain the @Autowired annotation and its different injection types.

@Autowired enables automatic dependency injection: Injection Types: 1. Constructor Injection (Recommended): ```java @Service public class ContentService { private final ContentRepository repository; @Autowired // Optional in Spring 4.3+ public ContentService(ContentRepository repository) { this.repository = repository; } } ``` 2. Setter Injection: ```java @Service public class ContentService { private ContentRepository repository; @Autowired public void setRepository(ContentRepository repository) { this.repository = repository; } } ``` 3. Field Injection: ```java @Service public class ContentService { @Autowired private ContentRepository repository; } ``` Why Constructor Injection is preferred: • Immutable dependencies • Easier testing (no setters needed) • Clear required dependencies • Prevents incomplete initialization @Autowired with Qualifiers: ```java @Autowired @Qualifier("mongoRepository") private ContentRepository repository; ``` @Required for mandatory dependencies (legacy): ```java @Autowired(required = true) private ContentRepository repository; ```

2.2 MVC Architecture in Spring

Question 1: Explain the MVC pattern and how Spring MVC implements it.

MVC (Model-View-Controller) separates application concerns: • Model: Data and business logic • View: Presentation layer • Controller: Handles user input and updates model/view Spring MVC Implementation: Model: • POJOs representing data • Service classes for business logic • Repository classes for data access View: • JSP, Thymeleaf templates • JSON responses for REST APIs • Static resources (CSS, JS) Controller: • @Controller classes • Handle HTTP requests • Return view names or data SMGO MVC Structure: ``` Controller Layer (@Controller/@RestController) ↓ Service Layer (@Service) ↓ Repository Layer (@Repository) ↓ Database (MongoDB) ``` Example: ```java @RestController @RequestMapping("/api/contents") public class ContentController { @Autowired private ContentService contentService; @GetMapping public List getAllContents() { return contentService.getAllContents(); } @PostMapping public Content createContent(@RequestBody Content content) { return contentService.createContent(content); } } ```

Question 2: What is the role of DispatcherServlet in Spring MVC?

DispatcherServlet is the front controller in Spring MVC, handling all HTTP requests: Responsibilities: 1. Request Reception: Receives all requests 2. Handler Mapping: Finds appropriate controller method 3. Handler Execution: Invokes controller with dependencies 4. View Resolution: Resolves view name to actual view 5. Response Rendering: Renders response to client Request Flow: ``` Client Request ↓ DispatcherServlet ↓ HandlerMapping (finds controller) ↓ HandlerAdapter (invokes controller) ↓ Controller Method Execution ↓ ViewResolver (resolves view) ↓ View Rendering ↓ Response to Client ``` Configuration in SMGO: ```java @Configuration public class WebConfig implements WebMvcConfigurer { @Override public void addCorsMappings(CorsRegistry registry) { registry.addMapping("/api/**") .allowedOrigins("http://localhost:4200") .allowedMethods("GET", "POST", "PUT", "DELETE"); } } ``` Benefits: • Centralized request handling • Clean separation of concerns • Extensible through interceptors • Automatic content negotiation

Question 3: Explain the difference between @Controller and @RestController.

@Controller vs @RestController: @Controller: • Returns view names (JSP, Thymeleaf) • Used for web applications with templates • Methods return String (view name) or ModelAndView • Requires @ResponseBody for JSON responses @RestController: • Returns data directly (JSON/XML) • @Controller + @ResponseBody combined • Used for REST APIs • Methods return domain objects, collections, ResponseEntity Examples: @Controller (Traditional web): ```java @Controller public class WebController { @GetMapping("/home") public String home(Model model) {

model.addAttribute("contents", contentService.getAll()); return "home"; // Returns home.jsp } @GetMapping("/api/contents") @ResponseBody public List getContents() { return contentService.getAll(); } } ``` @RestController (API): ```java @RestController @RequestMapping("/api") public class ApiController { @GetMapping("/contents") public List getContents() { return contentService.getAll(); // Returns JSON } @PostMapping("/contents") public ResponseEntity createContent(@RequestBody Content content) { Content saved = contentService.save(content); return ResponseEntity.created(uri).body(saved); } } ``` In SMGO: All controllers use @RestController for API endpoints

Question 4: How does Spring handle request mapping and parameter binding?

Spring maps requests and binds parameters through annotations: Request Mapping: ```java @RestController @RequestMapping("/api/contents") // Base path public class ContentController { @GetMapping("/{id}") // /api/contents/123 public Content getById(@PathVariable Long id) { return contentService.findById(id); } @GetMapping // /api/contents?page=1&size;=10 public Page getAll(@RequestParam(defaultValue = "0") int page, @RequestParam(defaultValue = "10") int size) { return contentService.findAll(PageRequest.of(page, size)); } @PostMapping // Body: {"title":"New","category":"Movie"} public Content create(@RequestBody Content content) { return contentService.save(content); } @PutMapping("/{id}") public Content update(@PathVariable Long id, @RequestBody Content content) { content.setId(id); return contentService.save(content); } } ``` Parameter Binding Types: • @PathVariable - URL path variables • @RequestParam - Query parameters • @RequestBody - Request body (JSON) • @RequestHeader - HTTP headers • @CookieValue - Cookie values • @ModelAttribute - Form data Advanced Binding: ```java @PostMapping("/search") public List search(@ModelAttribute SearchCriteria criteria) { // criteria.title, criteria.category, etc. } @GetMapping("/contents") public List getContents(@RequestParam(required = false) String category, @RequestParam(required = false) @DateTimeFormat(pattern = "yyyy-MM-dd") Date fromDate) { // Automatic type conversion } ```

3. SOFTWARE ENGINEERING

3.1 Project Architecture Analysis

Question 1: Analyze the SMGO project's architecture and identify the design patterns used.

SMGO Project Architecture Analysis: The SMGO (Show Match Go On) content management system implements a modern distributed architecture with multiple design patterns: Core Architecture Patterns: 1. **Layered Architecture**: Clear separation between presentation, business logic, and data layers 2. **Microservices Pattern**: Separate AI service, backend API, and frontend application 3. **Repository Pattern**: Abstract data access layer 4. **Service Layer Pattern**: Business logic encapsulation 5. **Observer Pattern**: Real-time notifications via WebSocket 6. **Strategy Pattern**: Multiple notification senders (Email, Firebase, WebSocket) 7. **Factory Pattern**: Content creation and service instantiation Technology Stack: • **Frontend**: Angular 21 (SPA with component architecture) • **Backend**: Spring Boot 3.2.3 (REST API with MVC) • **AI Service**: Flask/Python with XGBoost • **Database**: MongoDB (NoSQL document storage) • **Real-time**: WebSocket with STOMP protocol • **Notifications**: Firebase Cloud Messaging + Email fallback Key Design Patterns Identified: 1. **Repository Pattern**: ```java public interface ContentRepository extends MongoRepository { List findByCategory(String category); @Query("{'engagementScore': {$gte: ?0}}") List findHighEngagement(double minScore); } ``` 2. **Service Layer Pattern**: ```java @Service public class ContentAnalyticsService { public List getTop5Contents() { return contentRepository.findAll().stream() .sorted((a,b) -> Double.compare(calculateScore(b), calculateScore(a))) .limit(5) .collect(Collectors.toList()); } } ``` 3. **Observer Pattern** (WebSocket notifications): ```typescript export class NotificationSubject { private observers: NotificationObserver[] = []; attach(observer: NotificationObserver): void { this.observers.push(observer); } notify(notification: Notification): void { this.observers.forEach(observer => observer.update(notification)); } } ``` 4. **Strategy Pattern** (Multiple notification channels): ```java public interface NotificationStrategy { void send(Notification notification); } @Service @Qualifier("emailStrategy") public class EmailNotificationStrategy implements NotificationStrategy { @Override public void send(Notification notification) { // Email implementation } } @Service @Qualifier("firebaseStrategy") public class FirebaseNotificationStrategy implements NotificationStrategy { @Override public void send(Notification notification) { // Firebase implementation } } ``` Benefits: • **Testability:** Easy to mock repository in unit tests • **Flexibility:** Can switch from MongoDB to another database • **Separation of Concerns:** Business logic separated from data access • **Maintainability:** Changes to data layer don't affect business logic

Question 2: Explain the layered architecture implemented in the SMGO system.

SMGO implements a classic layered architecture with clear separation: 1. **Presentation Layer** (Angular Frontend): - Components: unified-home, ai-discovery, admin-notifications - Services: HTTP client, WebSocket client,

state management - Guards: Authentication and authorization - Pipes: Data transformation and formatting 2. **Application Layer** (Spring Boot Controllers): - REST controllers with request mapping - Input validation and error handling - Response formatting and HTTP status codes - Cross-origin resource sharing (CORS) 3. **Service Layer** (Business Logic): - ContentService: Content CRUD operations - NotificationService: Multi-channel notifications - UserService: Authentication and user management - AnalyticsService: Content engagement calculations 4. **Infrastructure Layer** (Data & External Services): - Repository layer: MongoDB data access - External APIs: AI recommendation service - Email service: SMTP notifications - Firebase: Push notifications - Scheduler: Automated tasks Layer Communication: ``` Angular Components ↓ HTTP/WebSocket Spring Controllers ↓ Method calls Service Classes ↓ Repository interfaces MongoDB Repositories ``` Benefits: • **Maintainability**: Changes isolated to specific layers • **Testability**: Each layer can be tested independently • **Scalability**: Layers can be scaled separately • **Flexibility**: Technology changes don't affect other layers Example layer interaction: ```typescript // Presentation Layer export class UnifiedHomeComponent { constructor(private contentService: ContentService) {} ngOnInit() { this.contentService.getTop5Contents().subscribe(contents => { this.topContents = contents; }); } } // Service Layer @Injectable() export class ContentService { constructor(private http: HttpClient) {} getTop5Contents(): Observable { return this.http.get('/api/contents/top5'); } } // Application Layer @RestController public class ContentController { @Autowired private ContentAnalyticsService analyticsService; @GetMapping("/top5") public List getTop5Contents() { return analyticsService.getTop5Contents(); } } // Infrastructure Layer @Repository public interface ContentRepository extends MongoRepository { // Only data access concerns Page findAll(Pageable pageable); List findByCategory(String category); } ```

Question 3: How does the SMGO project implement separation of concerns?

SMGO implements separation of concerns through multiple architectural patterns: 1. **Single Responsibility Principle (SRP)**: - Each service has one primary responsibility - Controllers only handle HTTP concerns - Repositories only handle data access - Components only handle UI concerns 2. **Domain-Driven Design (DDD)**: - Content domain with entities, value objects - User domain with authentication, profiles - Notification domain with channels, scheduling 3. **Cross-Cutting Concerns**: - Security handled by Spring Security - Logging handled by AOP aspects - Transactions handled by @Transactional - Caching handled by Spring Cache 4. **Infrastructure Separation**: - Database concerns isolated in repositories - External API calls isolated in services - UI concerns isolated in components - Business rules isolated in service methods Example separation: **HTTP Concerns** (Controller): ```java @RestController @RequestMapping("/api/contents") public class ContentController { @GetMapping public ResponseEntity> getContents(@RequestParam(defaultValue = "0") int page, @RequestParam(defaultValue = "10") int size) { try { List contents = contentService.getContents(page, size); return ResponseEntity.ok(contents); } catch (Exception e) { return ResponseEntity.status(500).build(); } } } ``` **Business Logic** (Service): ```java @Service public class ContentService { public List getContents(int page, int size) { // Business rules: filtering,

validation, calculations return contentRepository.findAll(PageRequest.of(page, size)) .filter(this::applyBusinessRules) .collect(Collectors.toList()); } } ``` **Data Access** (Repository): ```java @Repository public interface ContentRepository extends MongoRepository { // Only data access concerns Page findAll(Pageable pageable); List findByCategory(String category); } ``` **UI Concerns** (Component): ```typescript @Component({...}) export class ContentListComponent { // Only UI concerns: display, user interaction contents: Content[] = []; loading = false; onContentClick(content: Content) { this.router.navigate(['/content', content.id]); } } ```

Question 4: What microservices patterns are evident in the SMGO architecture?

SMGO exhibits several microservices patterns despite being a monolithic backend: 1. **Service Decomposition**: - **AI Service**: Separate Flask application for ML recommendations - **Backend API**: Spring Boot REST API - **Frontend**: Angular SPA - **Database**: MongoDB as separate data service 2. **API Gateway Pattern** (Conceptual): - Spring Boot acts as API gateway - Routes requests to appropriate services - Handles cross-cutting concerns (CORS, security) 3. **Service Discovery** (Simple): - Hardcoded service URLs in configuration - Could be enhanced with Eureka or Consul 4. **Circuit Breaker Pattern** (Implemented): - HTTP client with timeout and retry logic - Fallback mechanisms for AI service failures 5. **Saga Pattern** (Notification workflow): - Multi-step notification process - Compensation actions for failures 6. **Event-Driven Architecture**: - WebSocket for real-time notifications - Scheduled events for newsletters - Firebase push notifications 7. **Sidecar Pattern** (Potential): - Could add monitoring, logging sidecars Example microservices communication: **Synchronous Communication**: ```java @Service public class AiRecommendationService { @Autowired private RestTemplate restTemplate; public List getRecommendations(UserPreferences prefs) { try { ResponseEntity> response = restTemplate.postForEntity("http://localhost:5055/recommend", prefs, new ParameterizedTypeReference>() {}); return response.getBody(); } catch (Exception e) { // Circuit breaker: return default recommendations return getFallbackRecommendations(); } } } ``` **Asynchronous Communication**: ```java @Service public class NotificationService { @Autowired private SimpMessagingTemplate messagingTemplate; @Scheduled(fixedRate = 10000) // Every 10 seconds public void sendScheduledNotifications() { List pending = notificationRepository.findUnsent(); pending.forEach(notification -> { // Send via multiple channels sendViaWebSocket(notification); sendViaFirebase(notification); sendViaEmail(notification); }); } } ``` **Database per Service** (MongoDB collections): - Content collection - User collection - Notification collection - Analytics collection This architecture provides: • Independent deployment and scaling • Technology diversity (Java, Python, TypeScript) • Fault isolation • Team autonomy for different services

3.2 Scrum Methodology

Question 1: Explain the Scrum framework and its core roles, events, and artifacts.

Scrum is an agile framework for developing complex products: **Core Roles:** 1. **Product Owner**: Defines product vision, prioritizes backlog, accepts work 2. **Scrum Master**: Facilitates Scrum process, removes impediments, coaches team 3. **Development Team**: Cross-functional members who deliver potentially releasable increments **Core Events:** 1. **Sprint Planning**: Plan work for upcoming sprint (2-4 weeks) 2. **Daily Scrum**: 15-minute daily synchronization meeting 3. **Sprint Review**: Demonstrate increment and gather feedback 4. **Sprint Retrospective**: Inspect and adapt the process **Core Artifacts:** 1. **Product Backlog**: Ordered list of all desired work 2. **Sprint Backlog**: Work committed for current sprint 3. **Increment**: Sum of all completed product backlog items **Scrum Values:** • Commitment, Courage, Focus, Openness, Respect **Scrum Pillars:** • Transparency, Inspection, Adaptation SMGO Development with Scrum: **Product Backlog Items:** • As a user, I want AI-powered content recommendations • As an admin, I want real-time notification system • As a user, I want personalized newsletter • As a developer, I want automated testing **Sprint Structure:** • Sprint 1: Basic content management (CRUD) • Sprint 2: User authentication and profiles • Sprint 3: AI recommendation system • Sprint 4: Notification system • Sprint 5: Newsletter with web scraping • Sprint 6: Analytics and top 5 content

Question 2: How would you apply Scrum to develop the SMGO project?

Applying Scrum to SMGO development: **Product Backlog Creation:** 1. User Stories for core features 2. Technical stories for infrastructure 3. Spike stories for research (AI, Firebase) 4. Bug stories for issues **Sprint Planning Example (Sprint 1 - Foundations):** ``` Sprint Goal: Implement basic content management system Selected PBIs: • Create Content entity and MongoDB setup • Implement Content CRUD operations • Build basic Angular UI for content display • Set up Spring Boot REST API Sprint Backlog: • Backend: Content model, repository, service, controller • Frontend: Content list component, add/edit forms • Testing: Unit tests for services • Documentation: API documentation ``` **Daily Scrum Format:** • What did I do yesterday? • What will I do today? • Any impediments? **Sprint Review:** • Demo: Show working content CRUD • Feedback: UI improvements, API enhancements • Product Owner: Accepts completed features **Sprint Retrospective:** • What went well? (MongoDB setup was smooth) • What could be improved? (Need better testing practices) • Action items: Add code reviews, improve documentation **Scrum Board Example:** ``` To Do: • Implement AI recommendation algorithm • Set up Firebase notifications • Create newsletter scheduler In Progress: • Build recommendation service (John) • Design notification UI (Sarah) Done: • Content CRUD operations • User authentication • Basic UI components ``` **Definition of Done:** • Code written and unit tested • Code reviewed and approved • Acceptance criteria met • Documentation updated • Deployed to staging environment

Question 3: What are the challenges of implementing Scrum in a small development team?

Challenges of Scrum in small teams and solutions: **Challenge 1: Role Overlap** • Problem: One person may play multiple roles (PO + Developer) • Solution: Rotate roles or have external product owner • SMGO Solution: Developer acts as PO, external mentor as Scrum Master **Challenge 2: Limited Resources** • Problem: Small team can't work on many items simultaneously • Solution: Smaller sprint backlogs, focus on high-value items • SMGO Solution: 2-week sprints with 3-5 PBIs max **Challenge 3: Daily Standups** • Problem: May feel unnecessary with close communication • Solution: Keep brief (10-15 minutes), focus on impediments • SMGO Solution: Virtual standups with shared screen for progress **Challenge 4: Sprint Planning** • Problem: Estimating complex features (AI, real-time features) • Solution: Use story points, break down complex items • SMGO Solution: Spike stories for research before implementation **Challenge 5: Retrospectives** • Problem: Small team may lack diverse perspectives • Solution: Include stakeholders, use different retrospective formats • SMGO Solution: "What went well, what to improve, action items" **Challenge 6: Product Owner Availability** • Problem: PO may be busy with other responsibilities • Solution: Regular backlog refinement sessions • SMGO Solution: Weekly backlog grooming meetings **Challenge 7: Technical Debt** • Problem: Pressure to deliver features may ignore quality • Solution: Include technical debt in backlog, allocate time • SMGO Solution: 20% of sprint capacity for refactoring **Benefits for Small Teams:** • Fast feedback loops • Adaptable to changing requirements • Clear priorities and goals • Regular delivery of working software • Continuous improvement culture

Question 4: Explain the concept of technical debt and how to manage it in Scrum.

Technical debt is the accumulation of suboptimal code that needs to be refactored or rewritten: **Types of Technical Debt:** 1. **Code Debt**: Poor code quality, lack of tests 2. **Architecture Debt**: Outdated architecture patterns 3. **Documentation Debt**: Missing or outdated docs 4. **Testing Debt**: Insufficient test coverage 5. **Dependency Debt**: Outdated libraries **Causes in Scrum:** • Pressure to meet sprint goals • Lack of refactoring time • Skipping code reviews • Inadequate testing • Poor initial design decisions **Managing Technical Debt in Scrum:** 1. **Make it Visible:** • Add technical debt items to product backlog • Estimate effort required • Prioritize alongside features 2. **Allocate Time:** • "Technical Debt Sprints" every few sprints • 10-20% of sprint capacity for refactoring • Include in definition of done 3. **Prevention:** • Code reviews for all changes • Automated testing requirements • Architectural guidelines • Regular dependency updates 4. **Measurement:** • Code coverage metrics • Cyclomatic complexity • Technical debt ratio • Build stability metrics **SMGO Technical Debt Examples:** **Code Debt:** ```java // Bad: Tight coupling, no abstraction public class NotificationService { public void sendEmail(String message) { // Direct SMTP implementation // Hard to test, change, or extend } } // Better: Abstracted with interface public interface NotificationSender { void send(Notification notification); } ``` **Architecture Debt:** • Monolithic backend could be microservices • No caching layer for performance • Synchronous AI calls blocking UI **Testing Debt:** • Missing unit tests for services • No integration tests for API endpoints • Manual testing only **Documentation

Debt:** • Missing API documentation • No architecture decision records • Incomplete setup guides **Managing Debt in SMGO:** • Regular refactoring sessions • Code quality requirements • Technical debt backlog items • Sprint retrospectives to identify debt

3.3 Design Patterns

Question 1: Identify and explain the Gang of Four design patterns used in SMGO.

SMGO implements several Gang of Four (GoF) design patterns: **Creational Patterns:** 1. **Singleton Pattern** (Service Layer): ```java @Service public class ContentAnalyticsService { // Spring creates single instance private static final ContentAnalyticsService instance = new ContentAnalyticsService(); private ContentAnalyticsService() {} // Private constructor public static ContentAnalyticsService getInstance() { return instance; } } ``` 2. **Factory Pattern** (Notification Creation): ```java public class NotificationFactory { public static Notification createEmailNotification(String message) { return Notification.builder() .type(NotificationType.EMAIL) .message(message) .timestamp(LocalDateTime.now()) .build(); } } ``` **Structural Patterns:** 3. **Adapter Pattern** (External API Integration): ```java public interface AiServiceAdapter { List getRecommendations(UserPreferences prefs); } @Service public class FlaskAiAdapter implements AiServiceAdapter { @Override public List getRecommendations(UserPreferences prefs) { // Adapt Flask API to Java interface RestTemplate restTemplate = new RestTemplate(); return restTemplate.postForEntity("http://localhost:5055/recommend", prefs, List.class); } } ``` 4. **Decorator Pattern** (Content Enhancement): ```java public interface ContentProcessor { Content process(Content content); } @Service public class AnalyticsDecorator implements ContentProcessor { private final ContentProcessor processor; @Autowired public AnalyticsDecorator(@Qualifier("basicProcessor") ContentProcessor processor) { this.processor = processor; } @Override public Content process(Content content) { Content processed = processor.process(content); processed.setEngagementScore(calculateScore(processed)); return processed; } } ``` **Behavioral Patterns:** 5. **Observer Pattern** (Real-time Notifications): ```typescript export class NotificationSubject { private observers: NotificationObserver[] = []; attach(observer: NotificationObserver): void { this.observers.push(observer); } notify(notification: Notification): void { this.observers.forEach(observer => observer.update(notification)); } } ``` 6. **Strategy Pattern** (Multiple Notification Channels): ```java public interface NotificationStrategy { void send(Notification notification); } @Service @Qualifier("emailStrategy") public class EmailNotificationStrategy implements NotificationStrategy { @Override public void send(Notification notification) { // Email implementation } } @Service @Qualifier("firebaseStrategy") public class FirebaseNotificationStrategy implements NotificationStrategy { @Override public void send(Notification notification) { // Firebase implementation } } ``` 7. **Template Method Pattern** (Content Processing): ```java public abstract class ContentProcessorTemplate { public final Content processContent(Content content) { validate(content); enrich(content); save(content); return content; } protected abstract void validate(Content content); protected abstract void enrich(Content content); protected void save(Content content) { contentRepository.save(content); } } ``` 8. **Command Pattern** (Scheduled Tasks): ```java public interface Command { void execute(); } public class SendNewsletterCommand implements Command { @Override public void execute() { newsletterService.sendScheduledNewsletters(); } } @Service public class SchedulerService { @Scheduled(cron = "0 0 9 1 * ?") // Monthly public void executeMonthlyTasks() { Command newsletterCommand = new SendNewsletterCommand(); newsletterCommand.execute(); } } ```

Question 2: Explain how the Repository pattern is implemented in the SMGO project.

Repository Pattern implementation in SMGO: **Purpose:** Abstract data access layer, decouple business logic from data storage **Core Components:** 1. **Repository Interface:** ```java public interface ContentRepository extends MongoRepository { // Basic CRUD operations inherited from MongoRepository // Custom query methods List findByCategory(String category); List findByTitleContainingIgnoreCase(String title); // Custom queries with @Query @Query("{'viewCount': {$gte: ?0}}") List findPopularContents(int minViews); @Query("{'engagementScore': {$gte: ?0}}") List findHighEngagementContents(double minScore); // Aggregation queries @Aggregation(pipeline = { "{$match: {category: ?0}}", "{$sort: {engagementScore: -1}}", "{$limit: 5}" }) List findTop5ByCategory(String category); } ``` 2. **Service Layer using Repository:** ```java @Service public class ContentService { @Autowired private ContentRepository contentRepository; public List getAllContents() { return contentRepository.findAll(); } public List getContentsByCategory(String category) { return contentRepository.findByCategory(category); } public List getTop5Contents() { return contentRepository.findAll().stream() .sorted((a, b) -> Double.compare(calculateEngagementScore(b), calculateEngagementScore(a))) .limit(5) .collect(Collectors.toList()); } public Content saveContent(Content content) { content.setLastModified(LocalDateTime.now()); return contentRepository.save(content); } private double calculateEngagementScore(Content content) { return content.getViewCount() * 0.7 + content.getCommentsCount() * 3.0; } } ``` 3. **Controller using Service:** ```java @RestController @RequestMapping("/api/contents") public class ContentController { @Autowired private ContentService contentService; @GetMapping public List getAllContents() { return contentService.getAllContents(); } @GetMapping("/top5") public List getTop5Contents() { return contentService.getTop5Contents(); } @GetMapping("/category/{category}") public List getContentsByCategory(@PathVariable String category) { return contentService.getContentsByCategory(category); } } ``` **Benefits in SMGO:** • **Testability:** Easy to mock repository in unit tests • **Flexibility:** Can switch from MongoDB to another database • **Separation of Concerns:** Business logic separated from data access • **Maintainability:** Changes to data layer don't affect business logic **Spring Data Implementation:** • Automatic query generation from method names • @Query annotations for complex queries • Pagination and sorting support • Auditing capabilities (@CreatedDate, @LastModifiedDate)

Question 3: How does SMGO implement the Strategy pattern for notifications?

Strategy Pattern for notifications in SMGO: **Context:** Multiple notification channels (WebSocket, Firebase, Email) with different implementations **Strategy Interface:** ```java public interface NotificationStrategy { void send(Notification notification); boolean isAvailable(); NotificationChannel getChannel(); } ``` **Concrete Strategies:** 1. **WebSocket Strategy:** ```java @Service @Qualifier("websocketStrategy") public class

WebSocketNotificationStrategy implements NotificationStrategy { @Autowired private SimpMessagingTemplate messagingTemplate; @Override public void send(Notification notification) { messagingTemplate.convertAndSend("/topic/notifications", notification); } @Override public boolean isAvailable() { return true; // WebSocket always available internally } @Override public NotificationChannel getChannel() { return NotificationChannel.WEBSOCKET; } } ``` 2. **Firebase Strategy:** ```java @Service @Qualifier("firebaseStrategy") public class FirebaseNotificationStrategy implements NotificationStrategy { @Autowired private FirebaseMessaging firebaseMessaging; @Override public void send(Notification notification) { try { MulticastMessage message = MulticastMessage.builder() .setNotification(com.google.firebase.messaging.Notification.builder() .setTitle(notification.getTitle()) .setBody(notification.getMessage()) .build()) .addAllTokens(getUserTokens(notification.getUserId())) .build(); firebaseMessaging.sendMulticast(message); } catch (FirebaseMessagingException e) { throw new NotificationException("Firebase send failed", e); } } @Override public boolean isAvailable() { // Check Firebase service status return firebaseService.isHealthy(); } @Override public NotificationChannel getChannel() { return NotificationChannel.FIREBASE; } } ``` 3. **Email Strategy:** ```java @Service @Qualifier("emailStrategy") public class EmailNotificationStrategy implements NotificationStrategy { @Autowired private JavaMailSender mailSender; @Override public void send(Notification notification) { try { SimpleMailMessage message = new SimpleMailMessage(); message.setTo(getUserEmail(notification.getUserId())); message.setSubject(notification.getTitle()); message.setText(notification.getMessage()); mailSender.send(message); } catch (MailException e) { throw new NotificationException("Email send failed", e); } } @Override public boolean isAvailable() { // Check SMTP server connectivity return emailService.isHealthy(); } @Override public NotificationChannel getChannel() { return NotificationChannel.EMAIL; } } ``` **Context Class (Strategy Selector):** ```java @Service public class NotificationService { @Autowired private List strategies; public void sendNotification(Notification notification) { // Strategy selection based on priority and availability List availableStrategies = strategies.stream() .filter(NotificationStrategy::isAvailable) .sorted(this::byPriority) .collect(Collectors.toList()); // Send via all available channels availableStrategies.forEach(strategy -> { try { strategy.send(notification); log.info("Notification sent via {}", strategy.getChannel()); } catch (Exception e) { log.error("Failed to send via {}", strategy.getChannel(), e); } }); // Fallback: If no strategies available, queue for later if (availableStrategies.isEmpty()) { queueForRetry(notification); } } private int byPriority(NotificationStrategy a, NotificationStrategy b) { // Priority: WebSocket > Firebase > Email return Integer.compare(getPriority(a.getChannel()), getPriority(b.getChannel())); } private int getPriority(NotificationChannel channel) { switch (channel) { case WEBSOCKET: return 1; case FIREBASE: return 2; case EMAIL: return 3; default: return 4; } } } ``` **Benefits:** • **Extensibility:** Easy to add new notification channels • **Flexibility:** Can change strategies at runtime • **Testability:** Each strategy can be tested independently • **Maintainability:** Changes to one channel don't affect others **Usage in SMGO:** ```java // Immediate notification (real-time) notificationService.sendNotification(notification); // Scheduled notification (newsletter) @Scheduled(fixedRate = 10000) public void sendPendingNotifications() { List pending = notificationRepository.findUnsent(); pending.forEach(notificationService::sendNotification); } ```

Question 4: What other design patterns could be beneficial for SMGO's evolution?

Additional design patterns beneficial for SMGO evolution: **1. Command Pattern (Enhanced Scheduling):** ```java public interface Command { void execute(); void undo(); String getDescription(); } public class SendNewsletterCommand implements Command { @Override public void execute() { newsletterService.sendScheduledNewsletters(); } @Override public void undo() { newsletterService.cancelLastNewsletter(); } } // Command processor with queue public class CommandProcessor { private Queue commandQueue = new LinkedList<>(); public void addCommand(Command command) { commandQueue.add(command); } public void processCommands() { while (!commandQueue.isEmpty()) { commandQueue.poll().execute(); } } } ``` **2. Observer Pattern (Advanced Event System):** ```java public interface EventListener { void onContentCreated(Content content); void onUserRegistered(User user); } @Service public class EventBus { private Map, List> listeners = new HashMap<>(); public void subscribe(Class eventType, EventListener listener) { listeners.computeIfAbsent(eventType, k -> new ArrayList<>()).add(listener); } public void publish(Object event) { List eventListeners = listeners.get(event.getClass()); if (eventListeners != null) { eventListeners.forEach(listener -> { if (event instanceof Content) { listener.onContentCreated((Content) event); } }); } } } ``` **3. Circuit Breaker Pattern (External Service Resilience):** ```java public class CircuitBreaker { private State state = State.CLOSED; private int failureCount = 0; private long lastFailureTime = 0; public T execute(Supplier supplier, Supplier fallback) { if (state == State.OPEN) { if (System.currentTimeMillis() - lastFailureTime > TIMEOUT) { state = State.HALF_OPEN; } else { return fallback.get(); } } try { T result = supplier.get(); reset(); return result; } catch (Exception e) { recordFailure(); return fallback.get(); } } } // Usage for AI service public List getRecommendations(UserPreferences prefs) { return circuitBreaker.execute(() -> aiService.getRecommendations(prefs), () -> getCachedRecommendations(prefs)); } ``` **4. Builder Pattern (Complex Object Creation):** ```java public class ContentBuilder { private String id; private String title; private String category; private LocalDateTime createdDate; private List tags = new ArrayList<>(); public ContentBuilder id(String id) { this.id = id; return this; } public ContentBuilder title(String title) { this.title = title; return this; } public ContentBuilder category(String category) { this.category = category; return this; } public ContentBuilder tag(String tag) { this.tags.add(tag); return this; } public Content build() { Content content = new Content(); content.setId(id); content.setTitle(title); content.setCategory(category); content.setTags(tags); content.setCreatedDate(LocalDateTime.now()); return content; } } // Usage Content content = new ContentBuilder() .title("Inception") .category("Movie") .tag("Sci-Fi") .tag("Thriller") .build(); ``` **5. Proxy Pattern (Caching Layer):** ```java public interface ContentService { Content getContent(String id); } @Service public class CachedContentService implements ContentService { @Autowired private ContentService realService; @Autowired private CacheManager cacheManager; @Override public Content getContent(String id) { Content cached = cacheManager.get(id, Content.class); if (cached != null) { return cached; } Content content = realService.getContent(id); cacheManager.put(id, content); return content; } } ``` **6. State Pattern (Notification State Management):** ```java public interface NotificationState { void handle(NotificationContext context); } public class PendingState

implements NotificationState { @Override public void handle(NotificationContext context) { // Check if ready to send if (context.isReady()) { context.setState(new SendingState()); } } } public class NotificationContext { private NotificationState state; public NotificationContext() { this.state = new PendingState(); } public void setState(NotificationState state) { this.state = state; } public void process() { state.handle(this); } } ``` These patterns would enhance: • **Scalability:** Circuit breaker, caching • **Maintainability:** Builder, state patterns • **Reliability:** Circuit breaker, command pattern • **Extensibility:** Observer, strategy patterns